Outline

- Problems Caused by Redundancy
- Functional Dependencies (FDs)
- Normal Forms Based on Keys
- Decomposition
  - Lossless Join Decomposition
  - Dependency Preserving Decomposition
- **Normalization**
  - BCNF Decomposition Algorithm
  - **3NF Decomposition Algorithm**

3NF Definition (Review)

A relation R is in 3rd normal form if :

For every nontrivial dependency $A_1, A_2, ..., A_n \rightarrow B$ for R, $\{A_1, A_2, ..., A_n\}$ is a key for R, or B is part of a key

- Tradeoff:

  BCNF = no FD anomalies, but may lose some FDs

  3NF = keeps all FDs, but may have some anomalies

Canonical Cover

- An attribute of a functional dependency is said to be ***extraneous*** if we can remove it without changing the closure of the set of functional dependencies.
- **Extraneous attributes** is : Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.
  - **Removal from the left side**: Attribute **A** is extraneous in α if $A \in \alpha$ and **F** logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
  - **Removal from the right side**: Attribute **A** is extraneous in β if $A \in \beta$ and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies **F**.

Testing if an Attribute is Extraneous

- Consider a set F of functional dependencies and the functional dependency $f = \alpha \rightarrow \beta$ in F.
- To test if attribute **A** $\in \alpha = \mu$**A** is extraneous in $\alpha = \mu$**A**
  - *compute* $\mu^+ = (\{\alpha\} - \mathbf{A})^+$ using the dependencies in F
  - *check that* $\mu^+ = (\{\alpha\} - \mathbf{A})^+$ contains β; if it does, **A** is extraneous
- To test if attribute **B** $\in \beta = \nu$**B** is extraneous in $\beta = \nu$**B**
  - *compute* α^+ using only the dependencies in

$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - \mathbf{B})\}$$
$$= (F - \{f\}) \cup \{\alpha \rightarrow \nu\}$$

  - *check that* α^+ contains **B**; if it does, **B** is extraneous

Testing if an Attribute is Extraneous

- For example, suppose $F = \{ AB \rightarrow C, A \rightarrow BC, B \rightarrow C \}$. Is A extraneous in $AB \rightarrow C$?

  Under F, $B^+ = BC$

  B^+ includes C, so A is extraneous.

- For example, suppose $F = \{ AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$. Is C extraneous in $AB \rightarrow CD$?

  $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$

  Under F', $AB^+ = ABCDE$

  AB^+ includes C, so C is extraneous.

Canonical Cover

- A canonical cover **F_c** for **F** is a set of dependencies such that **F** logically implies all dependencies in **F_c**, and **F_c** logically implies all dependencies in **F**. Furthermore, **F_c** must have the following properties:

1. No functional dependency in F_c contains an **extraneous** attribute.
2. Each left side of a functional dependency in F_c is unique. That is, there are no two dependencies $\alpha 1 \rightarrow \beta 1$ and $\alpha 2 \rightarrow \beta 2$ in F_c such that **$\alpha 1 = \alpha 2$**.

Algorithm: Compute a Canonical Cover

- compute a canonical cover for F

repeat

replace any $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$

by $\alpha_1 \rightarrow \beta_1 \beta_2$

delete any **extraneous attribute**

from any $\alpha \rightarrow \beta$

until F does not change

Example: Canonical Cover

- Given: F = {E→A, E→B, A→BC, B→C}
- Canonical Cover F_c :
  - F = {E→A, E→B, A→BC, B→C}
  - F = {E→AB, A→BC, B→C}
  - F = {E→AB, A→BC, B→C}
  - F = {E→AB, A→B, B→C}
- So, F_c = {E→A, A→B, B→C}

Is C **extraneous** in A→BC**?**
F'={E→AB, A→B, B→C}
A^+=ABC
A^+ contains C, so C is extraneous.

Is B **extraneous** in E→AB **?**
F'={E→A, A→B, B→C}
E^+=ABCE
E^+ contains B, so B is extraneous.

Algorithm: 3NF Decomposition

find a canonical cover F_c for F

result = { }

for each $\alpha \rightarrow \beta$ in F_c do

  if no schema in result contains $\alpha\beta$ then

    add schema $\alpha\beta$ to result

end for

if no schema in result contains a candidate key for R

  choose any candidate key α for R

  add schema α to result (*ensure the lossless-join*)

end if

Example: 3NF Decomposition

- $R = (A,B,C,D,E,F,G)$
- $F = \{ A \rightarrow B, A \rightarrow C, D \rightarrow E, B \rightarrow A, F \rightarrow BG \}$
- $F_c = \{ A \rightarrow BC, D \rightarrow E, B \rightarrow A, F \rightarrow BG \}$
- Candidate key = DF

Example: 3NF Decomposition (Step1)

- $R = (A,B,C,D,E,F,G)$
- $F_c = \{ A \rightarrow BC,\ D \rightarrow E,\ B \rightarrow A,\ F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	
$D \rightarrow E$	
$B \rightarrow A$	
$F \rightarrow BG$	

Example: 3NF Decomposition (Step2)

- $R = (A,B,C,D,E,F,G)$
- $F_c = \{ A \rightarrow BC,\ D \rightarrow E,\ B \rightarrow A,\ F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	
$B \rightarrow A$	
$F \rightarrow BG$	

Example: 3NF Decomposition (Step3)

- R = (A,B,C,D,E,F,G)
- $F_c = \{ A \rightarrow BC,\ D \rightarrow E,\ B \rightarrow A,\ F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	

Example: 3NF Decomposition (Step4)

- R = (A,B,C,D,E,F,G)
- $F_c = \{ A \rightarrow BC, D \rightarrow E, B \rightarrow A, F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	

Example: 3NF Decomposition (Step5)

- $R = (A,B,C,D,E,F,G)$
- $F_c = \{ A \rightarrow BC,\ D \rightarrow E,\ B \rightarrow A,\ F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	

Example: 3NF Decomposition (Step6)

- R = (A,B,C,D,E,F,G)
- $F_c = \{ A \rightarrow BC,\ D \rightarrow E,\ B \rightarrow A,\ F \rightarrow BG \}$
- Candidate key = DF

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	FBG

Example: 3NF Decomposition (Step7)

- $R = (A,B,C,D,E,F,G)$
- $F_c = \{ A \rightarrow BC, D \rightarrow E, B \rightarrow A, F \rightarrow BG \}$
- Candidate key = DF
- Result = {ABC, DE, FBG}

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	FBG

Example: 3NF Decomposition (Step8)

- $R = (A,B,C,D,E,F,G)$
- $F_c = \{ A \rightarrow BC, D \rightarrow E, B \rightarrow A, F \rightarrow BG \}$
- Candidate key = DF
- Result = {ABC, DE, FBG, DF}

F_c	result
$A \rightarrow BC$	ABC
$D \rightarrow E$	DE
$B \rightarrow A$	
$F \rightarrow BG$	FBG

Example: 3NF Decomposition

- R = (A, B, C)
- F = { $A \rightarrow BC$, $B \rightarrow C$ }
- Candidate key = ?
- Is R 3NF? ?
- 3NF Decomposition:

?

3NF Decomposition

- 3NF decomposition guarantees that the decomposition is a *lossless-join*
- It guarantees that the decomposition is *dependency preserving*

BCNF vs. 3NF

- Always possible to decompose a relation into relations in 3NF and
  - the decomposition is lossless
  - dependencies are preserved
- Always possible to decompose a relation into relations in BCNF and
  - the decomposition is lossless
  - may not be possible to preserve dependencies

Design Goals

- Goal for a relational database design is:
  - BCNF
  - Lossless join
  - Dependency preservation
- If we cannot achieve this, we accept:
  - 3NF
  - Lossless join
  - Dependency preservation

End